

Récap TP — Attendus séance par séance

Owner	 louis reynouard
Tags	

Modalités d'évaluation

Le module donne lieu à deux notes de poids égal dans la moyenne finale.

Note 1 : Rendu Git (50%)

Un dépôt Git transmis en fin d'UE contenant les 5 notebooks et le rapport final.
Noté sur 100, ramené sur 20.

Livrable	Points
S1 : <code>S1_Ingestion_Stream_TRIGRAMME.dbc</code>	10
S2 : <code>S2_Silver_Stream_TRIGRAMME.dbc</code>	15
S3 : <code>S3_Qualite_Stream_TRIGRAMME.dbc</code>	20
S4 : <code>S4_Scoring_Stream_TRIGRAMME.dbc</code>	20
S5 : <code>S5_Final_TRIGRAMME.dbc</code> • rapport	35

Le S5 porte 35 points car il intègre le rapport de synthèse (challenge de volume, comparaison Spark Core, honnêteté technique) en plus du notebook.

Note 2 : Soutenance orale (50%)

15 minutes par étudiant ou binôme, note individuelle même pour les binômes.

7 minutes de démo live, 8 minutes de questions.

Pendant la démo, le pipeline doit déjà tourner sur le cluster. Vous montrerez:

- une table Delta qui grossit en temps réel
- les métriques Spark UI Structured Streaming
- la table `classified_stream_taxi` avec les labels K-Means

Un challenge sera imposé à la volée pendant ces 7 minutes : vous choisissez entre demander un passage en mode `chaos` pour vérifier la quarantaine en direct, ou imposer un `rate` élevé pour observer la Spark UI sous charge.

Les 8 minutes de questions suivent trois axes.

Axe	Poids
Démo live et réaction au challenge	30%
Décisions techniques (trigger, watermark, mode de sortie, checkpoint)	40%
Lien avec Spark Core (scoring K-Means, comparaison batch vs streaming)	30%

Moyenne finale

$$\text{Moyenne} = (\text{Note Git} / 20 + \text{Note Soutenance} / 20) / 2$$

Une note de soutenance inférieure à 6/20 peut être considérée comme éliminatoire indépendamment de la note Git.

Règles transversales (s'appliquent à tous les livrables)

Tout chemin de checkpoint doit pointer vers un Volume Unity Catalog persistant. Un checkpoint en `/tmp/` ou sur le filesystem local du cluster est invalide.

Tout trigger doit être justifié dans une cellule Markdown. La valeur par défaut sans explication n'est pas acceptée.

Chaque job streaming doit se terminer par un arrêt explicite via `.stop()`. Un notebook sans arrêt propre sera considéré incomplet.

Toutes les occurrences de `XYZ` dans les chemins doivent être remplacées par votre trigramme réel.

Un notebook sans cellules Markdown de documentation n'est pas accepté, quelle que soit la qualité du code.

Séance 1 : Premier pipeline streaming

Notebook attendu : `S1_Ingestion_Stream_TRIGRAMME.dbc`

Ce que vous devez produire.

Le simulateur configuré et lancé en mode `normal`. Chaque paramètre (rate, interval, duration) doit être justifié dans une cellule Markdown.

Un pipeline Bronze streaming via Auto Loader : ajout des colonnes `ingestion_timestamp` et `source_file`, écriture en table Delta en mode `append`. Trigger et checkpoint sont votre choix, à justifier.

Une preuve d'exécution : capture Spark UI onglet Structured Streaming montrant au moins 5 micro-batches avec le nombre de lignes par batch.

Une validation de cohérence de schéma entre `bronze_stream_taxi` et `bronze_taxi_trips` de Spark Core, avec commentaire sur les différences éventuelles.

Une cellule Markdown répondant à : pourquoi le mode `complete` est impraticable pour une table d'ingestion brute en streaming ? 5 lignes minimum, basées sur des arguments techniques.

Séance 2 : Silver Stream et transformations temporelles

Notebook attendu : `S2_Silver_Stream_TRIGRAMME.dbc`

Ce que vous devez produire.

Deux jobs streaming distincts avec deux checkpoints séparés : le job Silver (typepage, watermark, Stream-Static Join sur les zones) et le job KPI (agrégation par fenêtre de 10 minutes par borough). Le mode de sortie du job KPI doit être `update` avec une justification.

Le simulateur configuré avec `late_data_ratio: 0.15`. Le watermark du job Silver doit être paramétré en cohérence avec cette valeur, avec la justification du chiffre choisi.

La cellule de validation Bronze/Silver exécutée deux fois à des moments différents pendant la simulation. Les deux sorties figurent dans le notebook avec un

commentaire sur l'évolution du ratio.

Un `display()` de la table `kpi_stream_by_window` après au moins 15 minutes de simulation, montrant plusieurs fenêtres pour au moins deux boroughs.

Deux cellules Markdown distinctes répondant aux questions suivantes.

Question 1 : vous avez `late_data_ratio=0.15` et un watermark de 5 minutes. Que se passe-t-il si vous passez le watermark à 1 minute ? Quantifiez l'impact attendu sur le taux de données perdues dans votre contexte de simulation. Montrez le calcul.

Question 2 : pourquoi le mode `complete` serait-il dangereux pour `kpi_stream_by_window` sur un compte Databricks gratuit ? Estimez à partir de vos mesures combien de mémoire serait nécessaire après 24h de simulation en mode `complete`.

Séance 3 : Qualité en flux et détection d'anomalies

Notebook attendu : `S3_Qualite_Stream_TRIGRAMME.dbc`

Ce que vous devez produire.

Le simulateur configuré en mode `chaos` avec `chaos_ratio: 0.20`. Le pipeline ne doit pas crasher sur données corrompues : le routage se fait via la colonne `quality_flag`, jamais via une exception non gérée.

Deux jobs distincts avec deux checkpoints séparés : le job Gold (lignes valides) et le job Quarantaine (lignes rejetées). Deux tables Delta distinctes.

Le dashboard de qualité exécuté à deux moments différents pendant la simulation : taux de rejet global et distribution des types d'anomalies à chaque relevé. Les deux sorties figurent dans le notebook.

Le test de résilience : arrêt manuel du job Gold avec `query_gold.stop()`, relance du job, et preuve que les données ne sont pas dupliquées (count avant arrêt, count après relance, les deux identiques).

Deux cellules Markdown distinctes répondant aux questions suivantes.

Question 1 : vous observez que le taux de rejet dû à `ANOMALIE_STATISTIQUE` varie fortement d'un micro-batch à l'autre. Proposez une explication. Quel ajustement

permettrait de stabiliser ce taux ?

Question 2 : arguments pour ignorer définitivement les données en quarantaine versus les réinjecter après correction. Quelle serait votre décision dans le contexte taxi et pourquoi ?

Le test de résilience est éliminatoire : un pipeline qui ne survit pas à un restart ne sera pas noté.

Séance 4 : Scoring temps réel et détection de drift

Notebook attendu : `S4_Scoring_Stream_TRIGRAMME.dbc`

Ce que vous devez produire.

Une validation batch du modèle K-Means avant tout streaming :

`pipeline_model.transform()` sur 200 lignes, distribution des clusters affichée, comparaison commentée avec les centroïdes de Spark Core S4. Si votre modèle de Spark Core était de mauvaise qualité, dites-le explicitement et évaluez l'impact sur le scoring temps réel.

Le pipeline de scoring streaming : table `classified_stream_taxi` peuplée avec les colonnes `prediction` et `cluster_label`. Les labels doivent correspondre aux noms que vous aviez donnés à vos clusters en Spark Core S4.

La simulation en deux phases : 10 batches en mode `normal`, puis 10 batches en mode `airport_surge`. Les deux jobs (scoring + drift monitor) tournent pendant les deux phases.

L'analyse post-simulation de `drift_monitor` : tableau ou graphique fenêtres x clusters montrant l'évolution de la distribution. Le point de bascule entre les deux phases doit être identifiable visuellement.

Trois cellules Markdown répondant aux questions suivantes.

Question 1 : à partir de quelle fenêtre pouvez-vous affirmer avec certitude que la distribution a changé ? Quel critère statistique utilisez-vous et pourquoi ?

Question 2 : à quel seuil de drift faudrait-il réentraîner le modèle ? Comment mesureriez-vous ce seuil automatiquement ?

Question 3 (bonus) : pourquoi un modèle avec une inertie élevée peut tout de même produire un monitoring de drift fonctionnel ? Quelle est la limite de ce raisonnement ?

Séance 5 : Challenge de volume et livrable final

Notebook attendu : `S5_Final_TRIGRAMME.dbc`

Ce que vous devez produire.

Partie A. Testez les rates 20, 50, 100 et 200 : 10 batches par rate, Batch Duration relevée depuis la Spark UI. Remplissez un tableau de log dans le notebook. Tracez la courbe latence vs throughput avec une ligne horizontale au niveau de votre trigger interval. Identifiez et commentez le point de saturation.

Partie B. Appliquez au moins deux optimisations parmi : augmenter le trigger interval, mettre en cache les tables statiques, remplacer un Stream-Stream Join par un Stream-Static Join, appliquer `OPTIMIZE ZORDER` sur la table Gold. Mesurez la Batch Duration avant et après chaque optimisation. Si l'effet est faible ou nul, expliquez pourquoi.

Partie C. Un diagramme textuel ou image du pipeline complet (tables, jobs, checkpoints, flux). Les mêmes KPI que Spark Core S3 calculés sur vos données streaming et commentés. Une section "Ce qui ne fonctionne pas" d'au moins 10 lignes.